Notebook 8: Lorenz Equations

This notebook illustrates the training and prediction of a Neural Ordinary Differential Equation (NODEs) model. The data is fictitiously generated by the chaotic Lorenz Equation

$$\frac{dx}{dt} = 10(y - x) \tag{1}$$
$$\frac{dy}{dt} = x(28 - z) - y \tag{2}$$
$$\frac{dz}{dt} = xy - (8/3)z \tag{3}$$

with initial condition $x(0) = 10$, $y(0) = 10$ and $z(0) = 1$. Training data is generated for $t \in [0, 10]$. The intention is to try to use the NODEs model to predict the solution for $t \in [0, 20]$.

The NODEs model look like

$\frac{d\vec{u}}{dt} = \vec{NN}(\vec{u}; \vec{p})$

The Neural Network has 3wo inputs, u_1, u_2 and u_3 and three outputs du_1/dt, du_2/dt and du_3/dt.

In [1]:
```
using ComponentArrays
using Lux
using DiffEqFlux
using  OrdinaryDiffEq
using SciMLSensitivity
using  Optimization
using OptimizationOptimisers
using Random
using CairoMakie
using MLUtils
using OrdinaryDiffEqTsit5
```

Parameters of the training and validation data

In [2]:
```
u0 = Float32[10.0;10.0;10.0]
datasize = 2000
tspan = (0.0f0, 10.0f0)
tsteps = range(tspan[1], tspan[2]; length = datasize)
tspan2 = (0.0f0, 20.0f0)
tsteps2 = range(tspan2[1], tspan2[2]; length = 2000)
```

```
0.0f0:0.010005003f0:20.0f0
```

Training Parameters

In [3]:
```
max_iter_colloc=1000
max_iter_NODEs=2000
learning_rate=0.01
EpanechnikovKernelBandwidth=0.01
```

```
noise=0.00
mybatchsize_colloc=100
mybatchsize_NODEs=50
```

```
50
```

Define Lorenz Equation

$$\frac{du_1}{dt} = 10(u_2 - u_1) \tag{4}$$
$$\frac{du_2}{dt} = u_1(28 - u_3) - u_2 \tag{5}$$
$$\frac{du_3}{dt} = u_1 u_2 - (8/3)u_3 \tag{6}$$

In [4]:
```
function trueODEfunc(du, u, p, t)
    du[1]=10*(u[2]-u[1])
    du[2]=u[1]*(28-u[3])-u[2]
    du[3]=u[1]*u[2]-(8.0/3.0)*u[3]
end
```

```
trueODEfunc (generic function with 1 method)
```

Want to get on to the attractor first before getting the training data. Integragte for 50 time units before we start collecting data

In [5]:
```
#Initial integration to make sure we get onto the attractor
prob_trueode = ODEProblem(trueODEfunc, u0, (0.0f0, 50.0f0))
temp = Array(solve(prob_trueode, Tsit5()))
u0=temp[:,end]
```

```
3-element Vector{Float32}:
  -8.905455
 -13.592059
  19.998592
```

Generating training and validation data

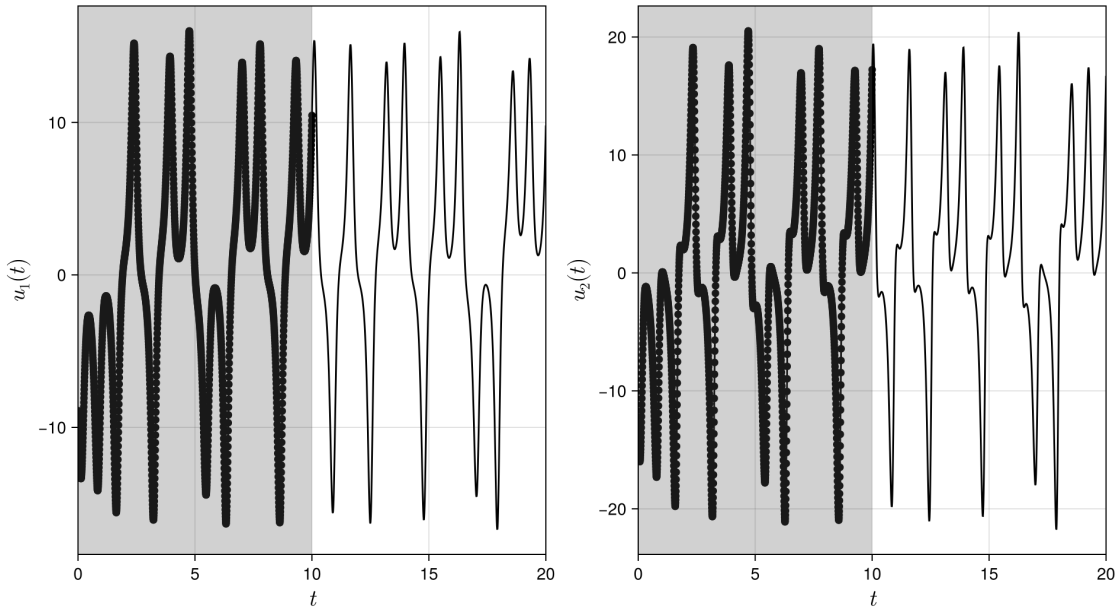
In [6]:
```
#This generates the training data
prob_trueode = ODEProblem(trueODEfunc, u0, tspan)
data = Array(solve(prob_trueode, Tsit5(); saveat = tsteps)) .+ noise*rand

#This is the prediction beyond the training data
prob_trueode_sol = ODEProblem(trueODEfunc, u0, tspan2)
data_true = Array(solve(prob_trueode_sol, Tsit5(); saveat = tsteps2))
```

```
3×2000 Matrix{Float32}:
  -8.90545   -9.37906   -9.85893  -10.3395  …   8.47334   9.12364   9.8023
5
 -13.5921   -14.1525   -14.6693   -15.1259     14.8112   15.7711   16.7093
  19.9986    20.7246    21.5483    22.4675     14.8869   15.8234   16.9244
```

Plot the raw data (without scaling)

In [7]:
```
fig = CairoMakie.Figure(size = (960, 540),backgroundcolor = :transparent)
ax1 = CairoMakie.Axis(fig[1, 1],
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
```

```
    xlabel=L"t",ylabel=L"u_1(t)")
CairoMakie.scatter!(ax1,tsteps, data[1,:];color=:black)
CairoMakie.lines!(ax1,tsteps2, data_true[1,:];color=:black)
xlims!(ax1,0,tspan2[2])
CairoMakie.vspan!(ax1,tspan[1], tspan[2];color=(:grey, 0.2))
#ylims!(ax1,0,1)

ax2 = CairoMakie.Axis(fig[1, 2],
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
    xlabel=L"t",ylabel=L"u_2(t)")
CairoMakie.scatter!(ax2,tsteps, data[2,:];color=:black)
CairoMakie.lines!(ax2,tsteps2, data_true[2,:];color=:black)
xlims!(ax2,0,tspan2[2])
CairoMakie.vspan!(ax2,tspan[1], tspan[2];color=(:grey, 0.2))
#ylims!(ax2,0,1)
fig
```



Normalize/Scale data to between 0 and 1

In [8]:
```
max_data=maximum(data,dims=2)
min_data=minimum(data,dims=2)

data=(data .- min_data) ./ (max_data .- min_data)
data_true=(data_true .- min_data) ./ (max_data .- min_data)
```

```
3×2000 Matrix{Float64}:
 0.2296    0.214951  0.200108  0.185242  …  0.767147  0.787262  0.808255
 0.18051   0.167044  0.154628  0.143658     0.862948  0.886011  0.908552
 0.313489  0.336677  0.362984  0.392345     0.150224  0.180137  0.215302
```

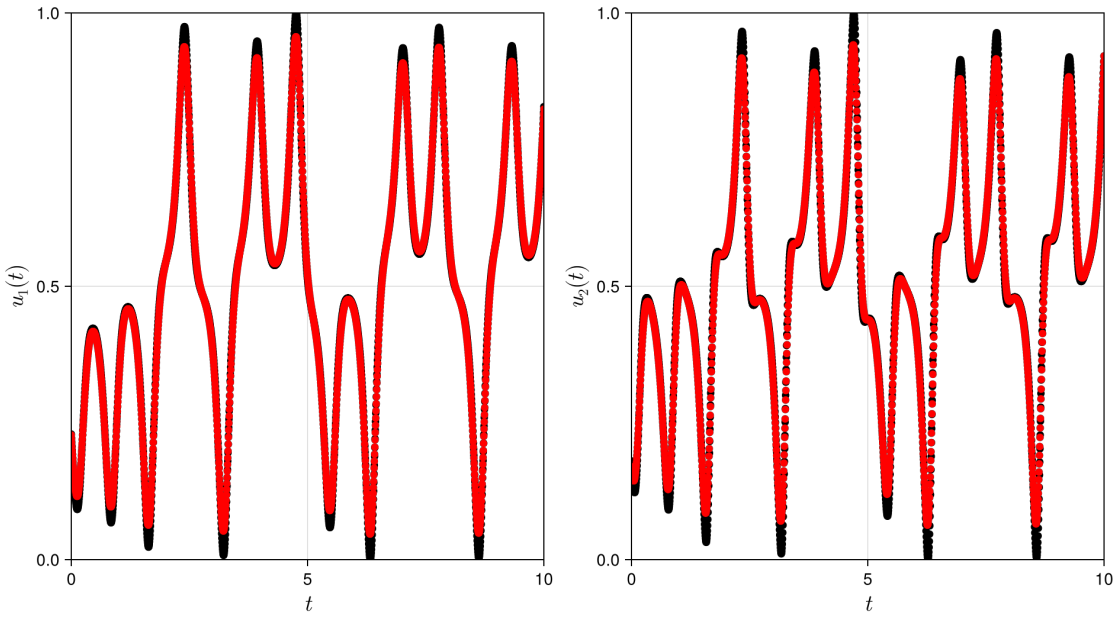
Smoothen data for collocation training

In [9]:
```
du, u = collocate_data(data, tsteps, EpanechnikovKernel(),EpanechnikovKer
```

```
([-1.6115640579057617 -1.603773267480669 … 2.1211945298310124 2.1672910101
469234; -1.6727689030975434 -1.5786630456851274 … 2.2065842423237654 2.200
5503873882835; 2.3729607810139 2.521901194618947 … 3.691138264126443 3.922
3539720214946], [0.22791009687635078 0.22054512925292438 … 0.8145715874213
328 0.8239580049073585; 0.17008598618431797 0.16519515282091557 … 0.910563
409246739 0.9213035009762968; 0.3005783680195361 0.316087344150948 … 0.239
34648855685953 0.2525830818990506])
```

Plot the smoothed data to check if it looks reasonable. Note that even though there are 3 variables in the Lorenz equation, only the first two variables will be plotted.

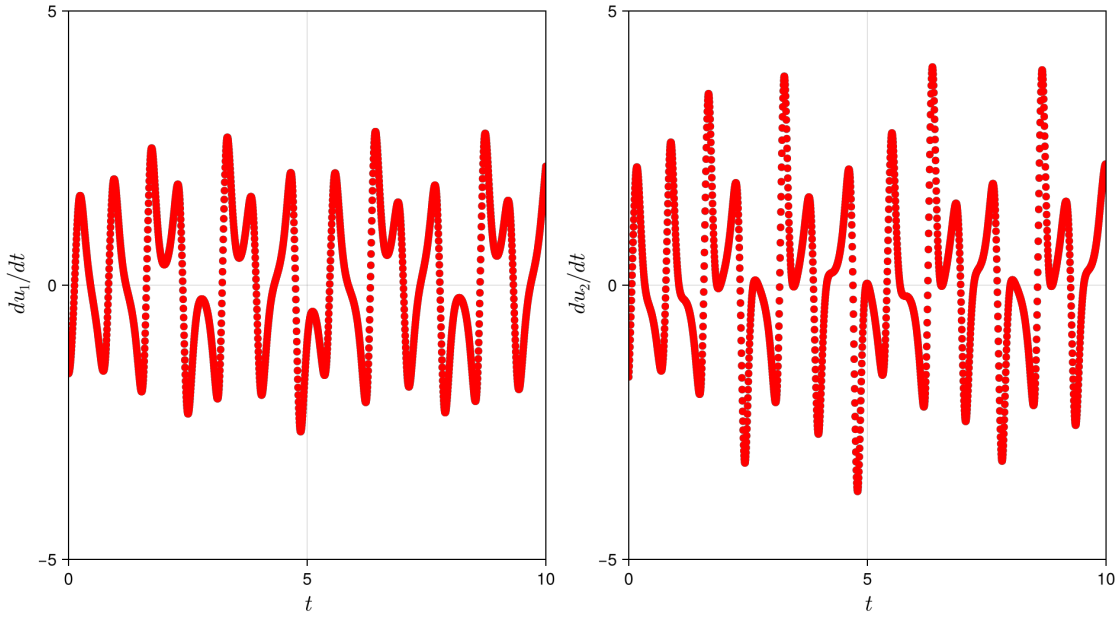
In [10]:
```
fig = CairoMakie.Figure(size = (960, 540),backgroundcolor = :transparent)
ax1 = CairoMakie.Axis(fig[1, 1],
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
    xlabel=L"t",ylabel=L"u_1(t)")
CairoMakie.scatter!(ax1,tsteps, data[1,:];color=:black)
CairoMakie.scatter!(ax1,tsteps, u[1,:];color=:red)
xlims!(ax1,0,tspan[2])
ylims!(ax1,0,1)

ax2 = CairoMakie.Axis(fig[1, 2],
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
    xlabel=L"t",ylabel=L"u_2(t)")
CairoMakie.scatter!(ax2,tsteps, data[2,:];color=:black)
CairoMakie.scatter!(ax2,tsteps, u[2,:]; color=:red)
xlims!(ax2,0,tspan[2])
ylims!(ax2,0,1)
save("colloc.png", fig)
fig
```



Plot the approximated derivative of the data to check if it looks reasonable

In [11]:
```
fig = CairoMakie.Figure(size = (960, 540),backgroundcolor = :transparent)

ax1 = CairoMakie.Axis(fig[1, 1],
```

```
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
    xlabel=L"t",ylabel=L"du_1/dt")
xlims!(ax1,0,tspan[2])
ylims!(ax1,-5,5)

ax2 = CairoMakie.Axis(fig[1, 2],
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
    xlabel=L"t",ylabel=L"du_2/dt")
xlims!(ax2,0,tspan[2])
ylims!(ax2,-5,5)

CairoMakie.scatter!(ax1,tsteps, du[1,:];color=:red)
CairoMakie.scatter!(ax2,tsteps, du[2,:];color=:red)
fig
```



Define and set up the Neural Network

In [12]:
```
rng = Xoshiro(0)
dudtNODEs = Chain(Dense(3, 5, tanh),
            Dense(5, 5, tanh),
             Dense(5, 3))
pinit, st = Lux.setup(rng, dudtNODEs)
```

```
((layer_1 = (weight = Float32[-0.04743576 1.207136 -0.54372525; -0.3143352
3 -0.76424813 -1.2789088; … ; -1.4381806 -0.6418321 -1.3020481; -0.9999523
0.5387089 -0.2807452], bias = Float32[0.1649376, -0.24305311, -0.14210694,
-0.5443836, -0.20890224]), layer_2 = (weight = Float32[1.0493203 -0.524454
3 … 0.9715817 -0.84806997; 0.08615455 0.9644778 … 0.94263804 0.34212336; …
; -0.91578704 0.38255343 … -0.70678467 -1.1776891; 0.71758914 -0.14458258
… -1.1133944 -0.48363888], bias = Float32[-0.28511116, -0.0044949492, -0.1
1264791, 0.31843108, 0.29759747]), layer_3 = (weight = Float32[0.1928618
0.20672485 … 0.31380573 -0.66837007; 0.28800184 -0.48583674 … 0.45046735
0.39924473; -0.26110658 -0.43949136 … 0.57463413 0.4578748], bias = Float3
2[-0.030889887, 0.095340915, 0.1666525])), (layer_1 = NamedTuple(), layer_
2 = NamedTuple(), layer_3 = NamedTuple()))
```

Setup loss_history() array to keep the loss values and also the callback function

In [13]:
```
# Initialize a vector to store loss values
loss_history = Float64[]

callback = function (p, l)
    if p.iter % 100 == 0
        println("Iteration: $(p.iter), Loss: $l")
    end
    push!(loss_history, l)
    return false
end
```

```
#15 (generic function with 1 method)
```

Collocation training

In [14]:
```
#######
# Collocation method training
################

function loss(p,(du_batch, u_batch))
    cost = zero(first(p))
    for i in 1:size(du_batch, 2)
        _du, _ = dudtNODEs(@view(u_batch[:, i]), p, st)
        dui = @view du_batch[:, i]
        cost += sum(abs2, _du .- dui)
    end
    return return 0.5*cost/size(du, 2)
end

dataloader_colloc = DataLoader((du,u); batchsize=mybatchsize_colloc,shuff
adtype = Optimization.AutoZygote()
optf = Optimization.OptimizationFunction((x, p) -> loss(x,(du,u)), adtype
optprob = Optimization.OptimizationProblem(optf, ComponentArray(pinit),da
result_neuralode = Optimization.solve(optprob, OptimizationOptimisers.Ada
```

```
┌ Warning: Mixed-Precision `matmul_cpu_fallback!` detected and Octavian.jl
cannot be used for this set of inputs (C [Matrix{Float64}]: A [Base.Reshap
edArray{Float32, 2, SubArray{Float32, 1, Vector{Float32}, Tuple{UnitRange
{Int64}}, true}, Tuple{}}] x B [Base.ReshapedArray{Float64, 2, SubArray{Fl
oat64, 1, Matrix{Float64}, Tuple{Base.Slice{Base.OneTo{Int64}}, Int64}, tr
ue}, Tuple{}}]). Converting to common type to to attempt to use BLAS. This
may be slow.
└ @ LuxLib.Impl /Users/asho/.julia/packages/LuxLib/zPBrt/src/impl/matmul.j
l:198
```

```
Iteration: 100, Loss: 3.591910887399705
Iteration: 200, Loss: 1.8302581073252717
Iteration: 300, Loss: 0.6818901074922887
Iteration: 400, Loss: 0.3931655564979314
Iteration: 500, Loss: 0.2791370228626781
Iteration: 600, Loss: 0.17002272223050555
Iteration: 700, Loss: 0.1251425076071947
Iteration: 800, Loss: 0.1046609466897008
Iteration: 900, Loss: 0.0930528196223559
Iteration: 1000, Loss: 0.08514957743537302
Iteration: 1100, Loss: 0.07878152535992915
Iteration: 1200, Loss: 0.07191856496467498
Iteration: 1300, Loss: 0.06573852209695191
Iteration: 1400, Loss: 0.06197142023386559
Iteration: 1500, Loss: 0.05818092316414544
Iteration: 1600, Loss: 0.055386934291741345
Iteration: 1700, Loss: 0.052998627231906105
Iteration: 1800, Loss: 0.05087763959960957
Iteration: 1900, Loss: 0.049086760206517255
Iteration: 2000, Loss: 0.047601473616716634
Iteration: 2100, Loss: 0.04581142765734739
Iteration: 2200, Loss: 0.04448402883631503
Iteration: 2300, Loss: 0.043118096059771724
Iteration: 2400, Loss: 0.04191206468295552
Iteration: 2500, Loss: 0.04079135774128798
Iteration: 2600, Loss: 0.040074259170083816
Iteration: 2700, Loss: 0.03869852630311939
Iteration: 2800, Loss: 0.03772559863661785
Iteration: 2900, Loss: 0.036800695040850824
Iteration: 3000, Loss: 0.03592958844977438
Iteration: 3100, Loss: 0.035029763308505484
Iteration: 3200, Loss: 0.034197082554051225
Iteration: 3300, Loss: 0.03338123162751523
Iteration: 3400, Loss: 0.032594042652014175
Iteration: 3500, Loss: 0.03184961086406583
Iteration: 3600, Loss: 0.03108335321223169
Iteration: 3700, Loss: 0.03037242983271828
Iteration: 3800, Loss: 0.029652281737760116
Iteration: 3900, Loss: 0.02902373283426631
Iteration: 4000, Loss: 0.02830381492186904
Iteration: 4100, Loss: 0.027730665405410325
Iteration: 4200, Loss: 0.029139657179506607
Iteration: 4300, Loss: 0.027229117005701395
Iteration: 4400, Loss: 0.02608887355087125
Iteration: 4500, Loss: 0.02914555018456598
Iteration: 4600, Loss: 0.024671044715812607
Iteration: 4700, Loss: 0.026367985475458256
Iteration: 4800, Loss: 0.02626908728557022
Iteration: 4900, Loss: 0.023035679435335434
Iteration: 5000, Loss: 0.0225389282283012
Iteration: 5100, Loss: 0.022162171253728647
Iteration: 5200, Loss: 0.021950235401209297
Iteration: 5300, Loss: 0.021098701216960752
Iteration: 5400, Loss: 0.020656134903317885
Iteration: 5500, Loss: 0.021692724066378396
Iteration: 5600, Loss: 0.01979013753351265
Iteration: 5700, Loss: 0.01938359023111945
Iteration: 5800, Loss: 0.019110185069857108
Iteration: 5900, Loss: 0.018607604282148933
Iteration: 6000, Loss: 0.018263163862262163
```

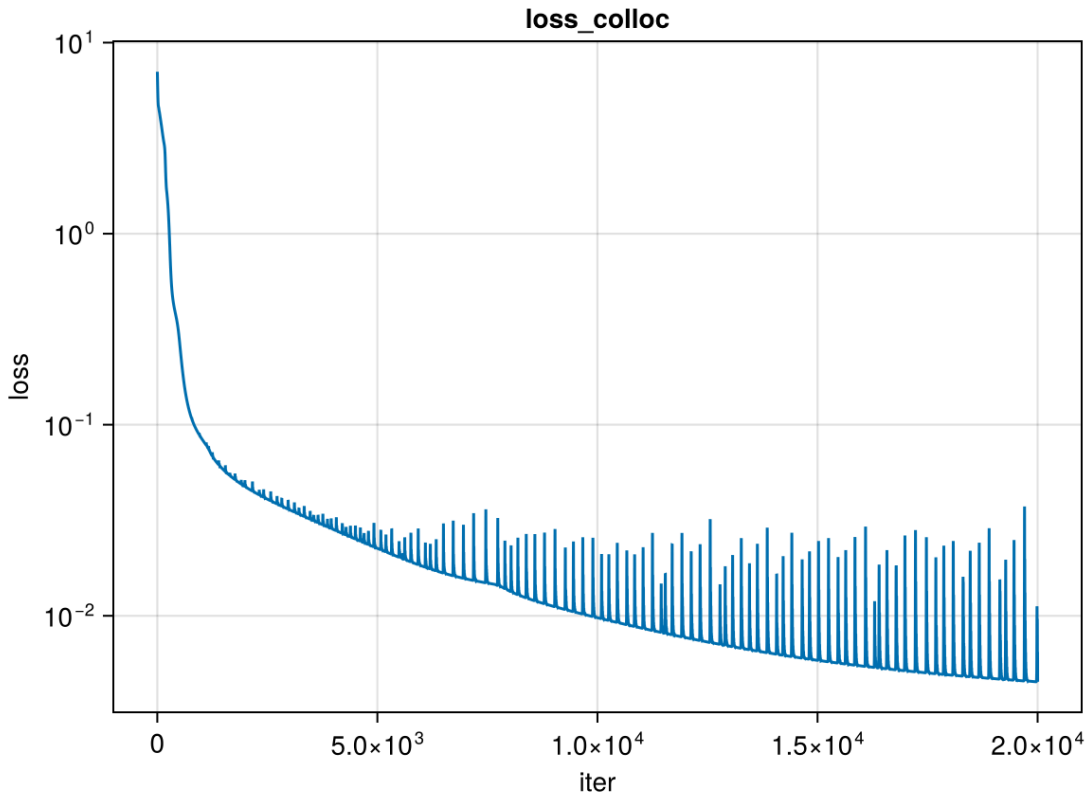
```
Iteration: 6100, Loss: 0.018535745937942046
Iteration: 6200, Loss: 0.0212392121501346
Iteration: 6300, Loss: 0.01728380893841339
Iteration: 6400, Loss: 0.017010070858476372
Iteration: 6500, Loss: 0.028250625728194884
Iteration: 6600, Loss: 0.016483441093912286
Iteration: 6700, Loss: 0.016236889247140996
Iteration: 6800, Loss: 0.01603666795560276
Iteration: 6900, Loss: 0.01582022684594308
Iteration: 7000, Loss: 0.01566839855369653
Iteration: 7100, Loss: 0.015449997304845716
Iteration: 7200, Loss: 0.016667341837748387
Iteration: 7300, Loss: 0.015127681350775085
Iteration: 7400, Loss: 0.014968305141408282
Iteration: 7500, Loss: 0.01498882313561867
Iteration: 7600, Loss: 0.014698795651329444
Iteration: 7700, Loss: 0.014508194864653634
Iteration: 7800, Loss: 0.014243255737100356
Iteration: 7900, Loss: 0.016031372433357362
Iteration: 8000, Loss: 0.013481537998067946
Iteration: 8100, Loss: 0.013170947565902816
Iteration: 8200, Loss: 0.01700955707973754
Iteration: 8300, Loss: 0.012604687945273557
Iteration: 8400, Loss: 0.012987195717673377
Iteration: 8500, Loss: 0.01214000959142184
Iteration: 8600, Loss: 0.01261295840459441
Iteration: 8700, Loss: 0.011738217000806335
Iteration: 8800, Loss: 0.015762837225272063
Iteration: 8900, Loss: 0.01137767770640011
Iteration: 9000, Loss: 0.011198770060890043
Iteration: 9100, Loss: 0.011053029121379685
Iteration: 9200, Loss: 0.010884670944055458
Iteration: 9300, Loss: 0.011019467326796015
Iteration: 9400, Loss: 0.010579283866483177
Iteration: 9500, Loss: 0.010519909768391766
Iteration: 9600, Loss: 0.010295502098921399
Iteration: 9700, Loss: 0.010438316356661141
Iteration: 9800, Loss: 0.010027804841626882
Iteration: 9900, Loss: 0.011260929953570089
Iteration: 10000, Loss: 0.009769431679905547
Iteration: 10100, Loss: 0.01348474713397684
Iteration: 10200, Loss: 0.009519624223304473
Iteration: 10300, Loss: 0.00944991255058496
Iteration: 10400, Loss: 0.00928106690739691
Iteration: 10500, Loss: 0.009235822351182534
Iteration: 10600, Loss: 0.00905502356515806
Iteration: 10700, Loss: 0.009291740242253483
Iteration: 10800, Loss: 0.008832615488645374
Iteration: 10900, Loss: 0.00875007763701923
Iteration: 11000, Loss: 0.008621310501544176
Iteration: 11100, Loss: 0.008535943639315298
Iteration: 11200, Loss: 0.008419417216225192
Iteration: 11300, Loss: 0.00834890125222201
Iteration: 11400, Loss: 0.008229797248668604
Iteration: 11500, Loss: 0.008131674200087782
Iteration: 11600, Loss: 0.008041200831718389
Iteration: 11700, Loss: 0.013297893925598273
Iteration: 11800, Loss: 0.007863128652979697
Iteration: 11900, Loss: 0.0077693421894599836
Iteration: 12000, Loss: 0.007695485484789991
```

```
Iteration: 12100, Loss: 0.007602556398600705
Iteration: 12200, Loss: 0.0075312310928568466
Iteration: 12300, Loss: 0.007441279166517023
Iteration: 12400, Loss: 0.007376750433145135
Iteration: 12500, Loss: 0.007288902961062177
Iteration: 12600, Loss: 0.00756269403196351
Iteration: 12700, Loss: 0.0071504488775999036
Iteration: 12800, Loss: 0.007128732677588897
Iteration: 12900, Loss: 0.013901847405427502
Iteration: 13000, Loss: 0.006935219432565753
Iteration: 13100, Loss: 0.007010721911042377
Iteration: 13200, Loss: 0.006801704664965263
Iteration: 13300, Loss: 0.006773213203067958
Iteration: 13400, Loss: 0.0066760507379644085
Iteration: 13500, Loss: 0.006675304709102891
Iteration: 13600, Loss: 0.006551651471811513
Iteration: 13700, Loss: 0.00650456403657427
Iteration: 13800, Loss: 0.006437665641856486
Iteration: 13900, Loss: 0.00644066284401181
Iteration: 14000, Loss: 0.006331920824382967
Iteration: 14100, Loss: 0.006468859521957815
Iteration: 14200, Loss: 0.006233177473905715
Iteration: 14300, Loss: 0.006177383758942539
Iteration: 14400, Loss: 0.006134144568403785
Iteration: 14500, Loss: 0.006079723691962831
Iteration: 14600, Loss: 0.00602581896980117
Iteration: 14700, Loss: 0.006037368930025735
Iteration: 14800, Loss: 0.0059446184459358495
Iteration: 14900, Loss: 0.00589541664655163
Iteration: 15000, Loss: 0.005843913286168448
Iteration: 15100, Loss: 0.005814842253261353
Iteration: 15200, Loss: 0.005761078826022668
Iteration: 15300, Loss: 0.005796982052127661
Iteration: 15400, Loss: 0.005682606612965916
Iteration: 15500, Loss: 0.0059588798018624265
Iteration: 15600, Loss: 0.005603296356365305
Iteration: 15700, Loss: 0.005576604421642502
Iteration: 15800, Loss: 0.0055303175536369755
Iteration: 15900, Loss: 0.005584141996517281
Iteration: 16000, Loss: 0.005462658665619833
Iteration: 16100, Loss: 0.005635817967188177
Iteration: 16200, Loss: 0.0053996781360857015
Iteration: 16300, Loss: 0.007763223847050746
Iteration: 16400, Loss: 0.01825983659582608
Iteration: 16500, Loss: 0.005300009439302513
Iteration: 16600, Loss: 0.00536916509346262
Iteration: 16700, Loss: 0.005239143600490636
Iteration: 16800, Loss: 0.00861152546398721
Iteration: 16900, Loss: 0.0051806079954636085
Iteration: 17000, Loss: 0.007399788013286801
Iteration: 17100, Loss: 0.005127468529403466
Iteration: 17200, Loss: 0.005095705816894139
Iteration: 17300, Loss: 0.005077618336838897
Iteration: 17400, Loss: 0.005046162769508798
Iteration: 17500, Loss: 0.00569321800774555
Iteration: 17600, Loss: 0.004997246716455666
Iteration: 17700, Loss: 0.00711311275547941
Iteration: 17800, Loss: 0.004946859938506657
Iteration: 17900, Loss: 0.005534993035908792
Iteration: 18000, Loss: 0.004900806901478126
```

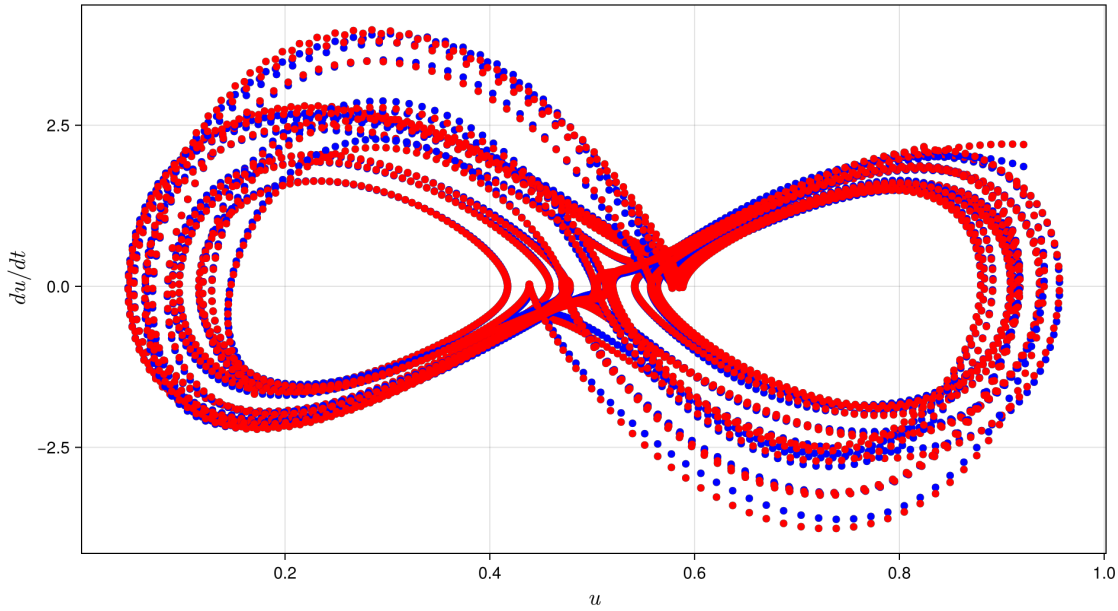
```
Iteration: 18100, Loss: 0.005242337444746369
Iteration: 18200, Loss: 0.004857119055836508
Iteration: 18300, Loss: 0.006294668179364201
Iteration: 18400, Loss: 0.004811667705344755
Iteration: 18500, Loss: 0.0052469461214967915
Iteration: 18600, Loss: 0.004768963670402224
Iteration: 18700, Loss: 0.005291807845888635
Iteration: 18800, Loss: 0.004729380254126358
Iteration: 18900, Loss: 0.022432065159648656
Iteration: 19000, Loss: 0.004693630235022328
Iteration: 19100, Loss: 0.00466895296113627
Iteration: 19200, Loss: 0.004664121208033602
Iteration: 19300, Loss: 0.004797450811852411
Iteration: 19400, Loss: 0.004612992846750453
Iteration: 19500, Loss: 0.0050249418867945456
Iteration: 19600, Loss: 0.004578468297277113
Iteration: 19700, Loss: 0.018304984359532362
Iteration: 19800, Loss: 0.004551258232007479
Iteration: 19900, Loss: 0.004528110681352435
Iteration: 20000, Loss: 0.0059600240519693025
Iteration: 20000, Loss: 0.004515236442831593
retcode: Default
u: ComponentVector{Float32}(layer_1 = (weight = Float32[2.179054 0.2784346
3 0.599759; -1.1563677 -0.70509815 0.2637689; … ; -1.4158007 0.47546118 -
0.8578029; -3.1087365 0.9291815 -0.02142499], bias = Float32[-0.6088147,
1.8094088, -0.092937544, -0.47029462, 2.4341292]), layer_2 = (weight = Flo
at32[-1.7843486 -1.8734723 … -1.3428993 0.7712734; 2.3968067 2.220555 … 2.
5754828 0.7122804; … ; -2.763579 -0.92833024 … -2.9315405 -4.4558825; -3.7
91065 0.07997521 … -0.5958464 1.4852889], bias = Float32[0.039563674, -0.8
9113694, 0.74663115, 3.0114417, 1.4815549]), layer_3 = (weight = Float32
[5.480252 1.4869664 … -6.4050393 -2.013455; -6.646257 2.3898003 … 4.397684
2.1934204; 1.7577549 -7.028674 … 1.9914551 1.0911086], bias = Float32[1.88
79006, -0.27045152, 1.2008728]))
```

Plot loss history

In [15]:
```
loss_history_collocation=copy(loss_history)
fig = CairoMakie.Figure()
ax = CairoMakie.Axis(fig[1, 1],yscale=log10,xlabel="iter",ylabel="loss",t
CairoMakie.lines!(ax,loss_history_collocation)
fig
```



In [16]:
```
du_pred,_=dudtNODEs(u, result_neuralode.u, st)
fig = CairoMakie.Figure(size = (960, 540), backgroundcolor = :transparent
ax = CairoMakie.Axis(fig[1, 1],
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
    xlabel=L"u",
    ylabel=L"du/dt")
CairoMakie.scatter!(ax,u[1,:],du_pred[1,:],marker=:circle,color=:blue)
CairoMakie.scatter!(ax,u[1,:],du[1,:],marker=:circle,color=:red)
CairoMakie.scatter!(ax,u[2,:],du_pred[2,:],marker=:circle,color=:blue)
CairoMakie.scatter!(ax,u[2,:],du[2,:],marker=:circle,color=:red)
save("result_collocation_training.png", fig)
fig
```



Predict using results of collocation training

In [17]:
```
u0=data_true[:,1]
```

```
3-element Vector{Float64}:
 0.22960015886601945
 0.18051000443333876
 0.3134886714756102
```

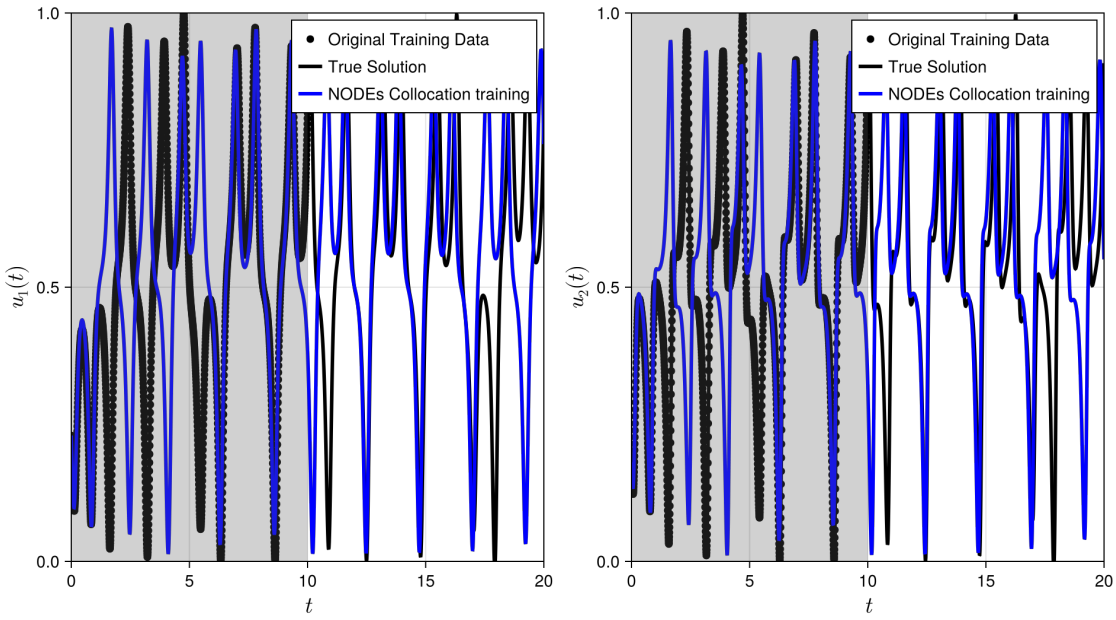
In [18]:
```
smodel_colloc = StatefulLuxLayer(dudtNODEs, result_neuralode, st)
dudt_colloc(u, p, t) = smodel_colloc(u, p)
prob_colloc = ODEProblem(dudt_colloc, u0, (tspan2[1], tspan2[2]),result_n
nn_sol_colloc = solve(prob_colloc, Tsit5(); sensealg=InterpolatingAdjoint
ComputedSolutionColloc=convert(AbstractArray,nn_sol_colloc)
```

```
retcode: Success
Interpolation: 1st order linear
t: 2000-element Vector{Float32}:
  0.0
  0.010005003
  0.020010006
  0.030015007
  0.04002001
  0.050025012
  0.060030013
  0.07003502
  0.08004002
  0.09004502
  ⋮
 19.91996
 19.929964
 19.93997
 19.949974
 19.95998
 19.969984
 19.97999
 19.989994
 20.0
u: 2000-element Vector{Vector{Float64}}:
 [0.22960015886601945, 0.18051000443333876, 0.3134886714756102]
 [0.214101448744511, 0.16777362271883298, 0.33882057420561307]
 [0.19844698805799765, 0.15629035576387817, 0.3675465936466725]
 [0.1828453694570498, 0.1465303163901175, 0.399515822354895]
 [0.16754927752835716, 0.1389931629695879, 0.43443266874911585]
 [0.15285467852587362, 0.13419583997405912, 0.47185389504543895]
 [0.13910698427279872, 0.13262742984290346, 0.5111630919922866]
 [0.1266878556471995, 0.1346937303760569, 0.5515837676451608]
 [0.11599570228432626, 0.14065550286053746, 0.5921963339450718]
 [0.10743563975880968, 0.15061992386276796, 0.6319566621114598]
 ⋮
 [0.9116845426432445, 0.7448965635687262, 0.8761819670012366]
 [0.8982592605693005, 0.7137652621929984, 0.8843610263290526]
 [0.8822745957382423, 0.6835065331943603, 0.8856901266331264]
 [0.8642731493505692, 0.6552421329242927, 0.8804566602169969]
 [0.844863791951906, 0.6291586423726208, 0.8697671633903207]
 [0.824477249896377, 0.6055175815762337, 0.8545806705345411]
 [0.8034733522278377, 0.5845137035863566, 0.8357621596866366]
 [0.782188433422288, 0.5662530675905372, 0.8141523852630421]
 [0.760903571371124, 0.5507210590763159, 0.7905399128394093]
```

In [19]:

```
smodel_colloc = StatefulLuxLayer(dudtNODEs, result_neuralode, st)
dudt_colloc(u, p, t) = smodel_colloc(u, p)
prob_colloc = ODEProblem(dudt_colloc, u0, (tspan2[1], tspan2[2]),result_n
nn_sol_colloc = solve(prob_colloc, Tsit5(); sensealg=InterpolatingAdjoint
ComputedSolutionColloc=convert(AbstractArray,nn_sol_colloc)

fig = CairoMakie.Figure(size = (960, 540), backgroundcolor = :transparent
ax1 = CairoMakie.Axis(fig[1, 1],
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
    xlabel=L"t",ylabel=L"u_1(t)")

CairoMakie.scatter!(ax1,tsteps, data[1,:];color=:black,label="Original Tr
CairoMakie.lines!(ax1,tsteps2, data_true[1,:]; linewidth = 3,color=:black
```

```
CairoMakie.lines!(ax1,nn_sol_colloc.t,ComputedSolutionColloc[1,:]; linewi
CairoMakie.vspan!(ax1,tspan[1], tspan[2];color=(:grey, 0.2))
axislegend(ax1;position=:rt)
xlims!(ax1,tspan2[1],tspan2[2])
ylims!(ax1,0,1)

ax2 = CairoMakie.Axis(fig[1, 2],
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
    xlabel=L"t",ylabel=L"u_2(t)")
CairoMakie.scatter!(ax2,tsteps, data[2,:];color=:black,label="Original Tr
CairoMakie.lines!(ax2,tsteps2, data_true[2,:]; linewidth = 3,color=:black
CairoMakie.lines!(ax2,nn_sol_colloc.t,ComputedSolutionColloc[2,:]; linewi
CairoMakie.vspan!(ax2,tspan[1], tspan[2];color=(:grey, 0.2))
axislegend(ax2;position=:rt)
xlims!(ax2,tspan2[1],tspan2[2])
ylims!(ax2,0,1)
fig
```



NODEs training

In [20]:

```
###################################
# "Ordinary Neural ODE training
###################################

struct TimeWrapper{T}
    t::T
end
MLDataDevices.isleaf(::TimeWrapper) = true
Base.length(t::TimeWrapper) = length(t.t)
Base.getindex(t::TimeWrapper, i) = TimeWrapper(t.t[i])
dataloader = DataLoader((data, TimeWrapper(tsteps)); batchsize=mybatchsiz

smodel = StatefulLuxLayer(dudtNODEs, result_neuralode, st)

function loss_adjoint(θ, (u_batch, t_batch))
    t_batch = t_batch.t
    u0 = u_batch[:, 1]
    dudt(u, p, t) = smodel(u, p)
```

```
    prob = ODEProblem(dudt, u0, (t_batch[1], t_batch[end]), θ)
    sol = solve(prob, Tsit5();  saveat=t_batch)
    pred = stack(sol.u)
    #temp=MSELoss()(pred, u_batch)
    #println(t_batch)
    return MSELoss()(pred, u_batch)
end

loss_history = Float64[]
opt_func = OptimizationFunction(loss_adjoint, Optimization.AutoZygote())
opt_prob = OptimizationProblem(opt_func, result_neuralode.u, dataloader)
numerical_neuralode = Optimization.solve(opt_prob, Optimisers.Adam(learni
loss_history_NODEs=copy(loss_history)
```

```
Iteration: 100, Loss: 0.002414470795438877
Iteration: 200, Loss: 0.0031199269863096224
Iteration: 300, Loss: 0.0025824422605247033
Iteration: 400, Loss: 0.0012415545740862368
Iteration: 500, Loss: 0.0005432154975871436
Iteration: 600, Loss: 0.0002982274187244451
Iteration: 700, Loss: 0.0009425503171858057
Iteration: 800, Loss: 0.001564973958595727
Iteration: 900, Loss: 0.0011851828251494524
Iteration: 1000, Loss: 0.0022371605549904405
Iteration: 1100, Loss: 0.0013633078524381572
Iteration: 1200, Loss: 0.0028819390300944053
Iteration: 1300, Loss: 0.0004941131300725224
Iteration: 1400, Loss: 0.0032493782831071122
Iteration: 1500, Loss: 0.0037237369803835903
Iteration: 1600, Loss: 0.0019620074175009062
Iteration: 1700, Loss: 0.0025203207985038906
Iteration: 1800, Loss: 0.0021922789138221835
Iteration: 1900, Loss: 0.004058847395542068
Iteration: 2000, Loss: 0.002842079135319057
Iteration: 2100, Loss: 0.0033778851793234456
Iteration: 2200, Loss: 0.0013696305393870712
Iteration: 2300, Loss: 0.003446071283027749
Iteration: 2400, Loss: 0.0034191095877587315
Iteration: 2500, Loss: 0.005222498567621716
Iteration: 2600, Loss: 0.0008177848162665976
Iteration: 2700, Loss: 0.0027351621099182422
Iteration: 2800, Loss: 0.0033725020607345865
Iteration: 2900, Loss: 0.006593254871973722
Iteration: 3000, Loss: 0.0003025585874309128
Iteration: 3100, Loss: 0.00548142320221526
Iteration: 3200, Loss: 0.0012289537915539787
Iteration: 3300, Loss: 0.006026172344978663
Iteration: 3400, Loss: 0.0011866279773589756
Iteration: 3500, Loss: 0.006463836051040088
Iteration: 3600, Loss: 0.0012645773104039766
Iteration: 3700, Loss: 0.0064722758096627505
Iteration: 3800, Loss: 0.0013024568135654223
Iteration: 3900, Loss: 0.006361152464123749
Iteration: 4000, Loss: 0.0012808350803863063
Iteration: 4100, Loss: 0.006072296051394776
Iteration: 4200, Loss: 0.001299950681399917
Iteration: 4300, Loss: 0.005730454975175496
Iteration: 4400, Loss: 0.0012872659850667414
Iteration: 4500, Loss: 0.005383919160662902
Iteration: 4600, Loss: 0.0012782800856282442
Iteration: 4700, Loss: 0.005037370029261173
Iteration: 4800, Loss: 0.001277519407084187
Iteration: 4900, Loss: 0.004720537468192382
Iteration: 5000, Loss: 0.0012789082651514583
Iteration: 5100, Loss: 0.004422302281154724
Iteration: 5200, Loss: 0.0012985107951519297
Iteration: 5300, Loss: 0.00416131800393543
Iteration: 5400, Loss: 0.001315437245247337
Iteration: 5500, Loss: 0.003917737927262905
Iteration: 5600, Loss: 0.001357470818553108
Iteration: 5700, Loss: 0.003717357228109072
Iteration: 5800, Loss: 0.0013825440432046047
Iteration: 5900, Loss: 0.003516267771511535
Iteration: 6000, Loss: 0.0014515469025957182
```

```
Iteration: 6100, Loss: 0.0033846344523170554
Iteration: 6200, Loss: 0.001464404928634887
Iteration: 6300, Loss: 0.003183850301100307
Iteration: 6400, Loss: 0.0015752077615587676
Iteration: 6500, Loss: 0.0031808465755284927
Iteration: 6600, Loss: 0.0015454006520403
Iteration: 6700, Loss: 0.002829303793725438
Iteration: 6800, Loss: 0.0013605804674128566
Iteration: 6900, Loss: 0.002666535114042581
Iteration: 7000, Loss: 0.002485272222281336
Iteration: 7100, Loss: 0.002733227888907865
Iteration: 7200, Loss: 0.011003802884202697
Iteration: 7300, Loss: 0.0022170829417515695
Iteration: 7400, Loss: 0.0003971883668468998
Iteration: 7500, Loss: 0.0024722623477356335
Iteration: 7600, Loss: 0.00022075074271276154
Iteration: 7700, Loss: 0.0017998629607477226
Iteration: 7800, Loss: 0.000543091551652339
Iteration: 7900, Loss: 0.001525640890646847
Iteration: 8000, Loss: 0.00027793996986626683
Iteration: 8100, Loss: 0.001557329989962031
Iteration: 8200, Loss: 0.0020848879459604
Iteration: 8300, Loss: 0.00263583275432118
Iteration: 8400, Loss: 0.0011532991607957393
Iteration: 8500, Loss: 0.002987576810795824
Iteration: 8600, Loss: 0.0013349865874242937
Iteration: 8700, Loss: 0.0030603234111500036
Iteration: 8800, Loss: 0.001348536609221573
Iteration: 8900, Loss: 0.002953993754049197
Iteration: 9000, Loss: 0.0013131741879641333
Iteration: 9100, Loss: 0.0029104267333691838
Iteration: 9200, Loss: 0.0016279388523117004
Iteration: 9300, Loss: 0.0028523115679240253
Iteration: 9400, Loss: 0.0004240200914442697
Iteration: 9500, Loss: 0.0044554772410115625
Iteration: 9600, Loss: 0.00038256410033877207
Iteration: 9700, Loss: 0.002382069627972945
Iteration: 9800, Loss: 0.00028436376300094974
Iteration: 9900, Loss: 0.0019958692894532907
Iteration: 10000, Loss: 0.00026477517169752195
Iteration: 10100, Loss: 0.0016349821894749291
Iteration: 10200, Loss: 0.0005616076413983271
Iteration: 10300, Loss: 0.0015477104168076965
Iteration: 10400, Loss: 0.00026439765813609286
Iteration: 10500, Loss: 0.0015110124893232396
Iteration: 10600, Loss: 0.0008215504812071141
Iteration: 10700, Loss: 0.002300389156691777
Iteration: 10800, Loss: 0.0010325141811357073
Iteration: 10900, Loss: 0.0024780606011035266
Iteration: 11000, Loss: 0.001064157914044593
Iteration: 11100, Loss: 0.0025868624089943185
Iteration: 11200, Loss: 0.0017921508686843716
Iteration: 11300, Loss: 0.002638925485689827
Iteration: 11400, Loss: 0.00035892971990499187
Iteration: 11500, Loss: 0.002549783282610306
Iteration: 11600, Loss: 0.0001740808040526888
Iteration: 11700, Loss: 0.0020982411778464843
Iteration: 11800, Loss: 0.0002493365617288726
Iteration: 11900, Loss: 0.0018899866951658936
Iteration: 12000, Loss: 0.00026465289577931833
```

```
Iteration: 12100, Loss: 0.0014942586623402125
Iteration: 12200, Loss: 0.0002370856464116317
Iteration: 12300, Loss: 0.0018754997858166666
Iteration: 12400, Loss: 0.0004483147040504 1015
Iteration: 12500, Loss: 0.0019775692556484657
Iteration: 12600, Loss: 0.0035581692673248527
Iteration: 12700, Loss: 0.0031176348088718608
Iteration: 12800, Loss: 0.0005402398343949183
Iteration: 12900, Loss: 0.0014642286023109881
Iteration: 13000, Loss: 0.0027555428218884906
Iteration: 13100, Loss: 0.002778503551529374
Iteration: 13200, Loss: 0.00036566053006455406
Iteration: 13300, Loss: 0.0013800191395904541
Iteration: 13400, Loss: 0.0009899193460830575
Iteration: 13500, Loss: 0.0020894014799423457
Iteration: 13600, Loss: 0.00030109405513428875
Iteration: 13700, Loss: 0.0017510279139957422
Iteration: 13800, Loss: 0.0008981746324679341
Iteration: 13900, Loss: 0.0017103309479478868
Iteration: 14000, Loss: 0.0008147056244616812
Iteration: 14100, Loss: 0.001990673802146936
Iteration: 14200, Loss: 0.0001701861054419678
Iteration: 14300, Loss: 0.0017174374775833171
Iteration: 14400, Loss: 0.0033780116946990043
Iteration: 14500, Loss: 0.002763464095600987
Iteration: 14600, Loss: 0.0008376044964162632
Iteration: 14700, Loss: 0.0014345656028455361
Iteration: 14800, Loss: 0.0034094512056661703
Iteration: 14900, Loss: 0.0024555718577420047
Iteration: 15000, Loss: 0.000619655509420945
Iteration: 15100, Loss: 0.0013458193358251629
Iteration: 15200, Loss: 0.003290403072262986
Iteration: 15300, Loss: 0.00231486709260988
Iteration: 15400, Loss: 0.0006140385682039832
Iteration: 15500, Loss: 0.0013002022565003152
Iteration: 15600, Loss: 0.0032075896122655072
Iteration: 15700, Loss: 0.0022042533990313663
Iteration: 15800, Loss: 0.0006099673588147401
Iteration: 15900, Loss: 0.0012726063866079593
Iteration: 16000, Loss: 0.0031547547389581894
Iteration: 16100, Loss: 0.002104436887124319
Iteration: 16200, Loss: 0.0006079032147640406
Iteration: 16300, Loss: 0.0012535151545486664
Iteration: 16400, Loss: 0.0031008124510560485
Iteration: 16500, Loss: 0.002008043093251377
Iteration: 16600, Loss: 0.0006081807731794904
Iteration: 16700, Loss: 0.001238589826479378
Iteration: 16800, Loss: 0.0030385693761947647
Iteration: 16900, Loss: 0.0019131961267806067
Iteration: 17000, Loss: 0.0006100815592726771
Iteration: 17100, Loss: 0.0012265611677470085
Iteration: 17200, Loss: 0.002965967072501724
Iteration: 17300, Loss: 0.0018199799819694426
Iteration: 17400, Loss: 0.0006129113148098494
Iteration: 17500, Loss: 0.0012175524118975417
Iteration: 17600, Loss: 0.00288320242173591
Iteration: 17700, Loss: 0.001728861373414425
Iteration: 17800, Loss: 0.0006162448296825794
Iteration: 17900, Loss: 0.001211417132088453
Iteration: 18000, Loss: 0.002789979738890503
```

```
Iteration: 18100, Loss: 0.0016413446562455185
Iteration: 18200, Loss: 0.000620086068103192
Iteration: 18300, Loss: 0.001207418917922019
Iteration: 18400, Loss: 0.0026872745451149154
Iteration: 18500, Loss: 0.0015584921251265967
Iteration: 18600, Loss: 0.0006251605085785771
Iteration: 18700, Loss: 0.0012043235705482339
Iteration: 18800, Loss: 0.0025769260342202558
Iteration: 18900, Loss: 0.0014782860721035814
Iteration: 19000, Loss: 0.0006311373822450973
Iteration: 19100, Loss: 0.0012025280859912142
Iteration: 19200, Loss: 0.0024625813742676778
Iteration: 19300, Loss: 0.001399540146035706
Iteration: 19400, Loss: 0.0006370418517708843
Iteration: 19500, Loss: 0.0012026510361925922
Iteration: 19600, Loss: 0.0023482665135710285
Iteration: 19700, Loss: 0.0013229990882514403
Iteration: 19800, Loss: 0.0006417003416358022
Iteration: 19900, Loss: 0.0012046286599800436
Iteration: 20000, Loss: 0.0022338537984068064
Iteration: 20100, Loss: 0.001251544234139524
Iteration: 20200, Loss: 0.0006464347926114775
Iteration: 20300, Loss: 0.00120774148485088
Iteration: 20400, Loss: 0.0021208114315564614
Iteration: 20500, Loss: 0.0011848028384208178
Iteration: 20600, Loss: 0.0006512585700097798
Iteration: 20700, Loss: 0.0012132614964032772
Iteration: 20800, Loss: 0.002009692779383219
Iteration: 20900, Loss: 0.0011213036515054817
Iteration: 21000, Loss: 0.0006555241253247218
Iteration: 21100, Loss: 0.001219214322007162
Iteration: 21200, Loss: 0.0019022755322574093
Iteration: 21300, Loss: 0.0010605233392602302
Iteration: 21400, Loss: 0.0006601709131724348
Iteration: 21500, Loss: 0.0012243528772954003
Iteration: 21600, Loss: 0.0018015388493478177
Iteration: 21700, Loss: 0.0010032353546099218
Iteration: 21800, Loss: 0.0006644841228073561
Iteration: 21900, Loss: 0.0012260888387732795
Iteration: 22000, Loss: 0.0017081210202866133
Iteration: 22100, Loss: 0.0009504550033815327
Iteration: 22200, Loss: 0.0006682506273083845
Iteration: 22300, Loss: 0.0012257843213006924
Iteration: 22400, Loss: 0.0016211319095416371
Iteration: 22500, Loss: 0.000902065165176578
Iteration: 22600, Loss: 0.0006716882467552874
Iteration: 22700, Loss: 0.0012237728923907697
Iteration: 22800, Loss: 0.0015402916954122888
Iteration: 22900, Loss: 0.0008572337917130434
Iteration: 23000, Loss: 0.00067551354773061
Iteration: 23100, Loss: 0.0012193481024920232
Iteration: 23200, Loss: 0.0014661859764684975
Iteration: 23300, Loss: 0.0008161616687548836
Iteration: 23400, Loss: 0.0006778760250596117
Iteration: 23500, Loss: 0.00121205026939508
Iteration: 23600, Loss: 0.001397870837871513
Iteration: 23700, Loss: 0.000778292510712427
Iteration: 23800, Loss: 0.0006801844953206648
Iteration: 23900, Loss: 0.001201983580136684
Iteration: 24000, Loss: 0.0013349006258213557
```

```
Iteration: 24100, Loss: 0.0007437391069608879
Iteration: 24200, Loss: 0.0006819073310143001
Iteration: 24300, Loss: 0.0011893520971324888
Iteration: 24400, Loss: 0.0012780153765543046
Iteration: 24500, Loss: 0.0007122007745022299
Iteration: 24600, Loss: 0.0006833049900545577
Iteration: 24700, Loss: 0.0011739839530790523
Iteration: 24800, Loss: 0.0012255025347072423
Iteration: 24900, Loss: 0.0006833047645650333
Iteration: 25000, Loss: 0.0006840591242036732
Iteration: 25100, Loss: 0.0011559834459274354
Iteration: 25200, Loss: 0.0011771606723739749
Iteration: 25300, Loss: 0.0006568793200699407
Iteration: 25400, Loss: 0.0006841720902588998
Iteration: 25500, Loss: 0.0011362396713262503
Iteration: 25600, Loss: 0.0011325770266348602
Iteration: 25700, Loss: 0.0006325104209140168
Iteration: 25800, Loss: 0.0006840249390022527
Iteration: 25900, Loss: 0.001115291505876559
Iteration: 26000, Loss: 0.0010910917306687822
Iteration: 26100, Loss: 0.0006100658760762307
Iteration: 26200, Loss: 0.0006841204433689276
Iteration: 26300, Loss: 0.001093221767763575
Iteration: 26400, Loss: 0.0010519491394374463
Iteration: 26500, Loss: 0.0005890175246568975
Iteration: 26600, Loss: 0.0006838761446804458
Iteration: 26700, Loss: 0.001070407551838512
Iteration: 26800, Loss: 0.001015263708261891
Iteration: 26900, Loss: 0.000568705243458701
Iteration: 27000, Loss: 0.0006826334519154255
Iteration: 27100, Loss: 0.0010477298832339938
Iteration: 27200, Loss: 0.0009805661138517978
Iteration: 27300, Loss: 0.0005482632555572276
Iteration: 27400, Loss: 0.0006809015969637668
Iteration: 27500, Loss: 0.00102552727230519
Iteration: 27600, Loss: 0.0009479640138791722
Iteration: 27700, Loss: 0.000528183750348432
Iteration: 27800, Loss: 0.0006789566406289237
Iteration: 27900, Loss: 0.0010042735230877
Iteration: 28000, Loss: 0.0009166059545181313
Iteration: 28100, Loss: 0.0005083415093642673
Iteration: 28200, Loss: 0.0006760794486221346
Iteration: 28300, Loss: 0.0009835831102952845
Iteration: 28400, Loss: 0.000886900177575841
Iteration: 28500, Loss: 0.0004888563959255533
Iteration: 28600, Loss: 0.0006727925889218979
Iteration: 28700, Loss: 0.00096432338440173
Iteration: 28800, Loss: 0.0008585554861115013
Iteration: 28900, Loss: 0.0004695552405570886
Iteration: 29000, Loss: 0.00066890904628703
Iteration: 29100, Loss: 0.0009452516310359653
Iteration: 29200, Loss: 0.0008322804835891357
Iteration: 29300, Loss: 0.0004506813232904355
Iteration: 29400, Loss: 0.0006641390607424661
Iteration: 29500, Loss: 0.0009270881137063938
Iteration: 29600, Loss: 0.000807407164050236
Iteration: 29700, Loss: 0.00043232121318426733
Iteration: 29800, Loss: 0.0006588618292837103
Iteration: 29900, Loss: 0.0009096619042676836
Iteration: 30000, Loss: 0.0007834752789110361
```

```
Iteration: 30100, Loss: 0.00041444823227344466
Iteration: 30200, Loss: 0.0006527386206586546
Iteration: 30300, Loss: 0.000893304401027779
Iteration: 30400, Loss: 0.00059948946027852
Iteration: 30500, Loss: 0.0003973669874124356
Iteration: 30600, Loss: 0.0006459695381185995
Iteration: 30700, Loss: 0.0008776115051541974
Iteration: 30800, Loss: 0.0007373398065967128
Iteration: 30900, Loss: 0.00038111862890153654
Iteration: 31000, Loss: 0.0006391535303905351
Iteration: 31100, Loss: 0.0008626543216480618
Iteration: 31200, Loss: 0.0007142943537333189
Iteration: 31300, Loss: 0.0003659615366487547
Iteration: 31400, Loss: 0.0006316350203708814
Iteration: 31500, Loss: 0.0008486182094847459
Iteration: 31600, Loss: 0.000691764375113811
Iteration: 31700, Loss: 0.00035160562097915737
Iteration: 31800, Loss: 0.0006242418958077425
Iteration: 31900, Loss: 0.000835123364060085
Iteration: 32000, Loss: 0.0006687249746522178
Iteration: 32100, Loss: 0.0003383658134810528
Iteration: 32200, Loss: 0.0006159632161482363
Iteration: 32300, Loss: 0.0008228351645760367
Iteration: 32400, Loss: 0.0006458173937021741
Iteration: 32500, Loss: 0.00032587770337566833
Iteration: 32600, Loss: 0.0006071174910563207
Iteration: 32700, Loss: 0.0008115003610814746
Iteration: 32800, Loss: 0.000622880990206453
Iteration: 32900, Loss: 0.00031424501272259224
Iteration: 33000, Loss: 0.0005978498928969179
Iteration: 33100, Loss: 0.000801062490816618
Iteration: 33200, Loss: 0.0005999730842944281
Iteration: 33300, Loss: 0.0003035980606758443
Iteration: 33400, Loss: 0.0005875597595404594
Iteration: 33500, Loss: 0.0007916099283897814
Iteration: 33600, Loss: 0.0005774979169562152
Iteration: 33700, Loss: 0.00029380552790041615
Iteration: 33800, Loss: 0.0005765833703501369
Iteration: 33900, Loss: 0.0007828187406063261
Iteration: 34000, Loss: 0.000555333029323532
Iteration: 34100, Loss: 0.00028471345718824014
Iteration: 34200, Loss: 0.0005648083735927939
Iteration: 34300, Loss: 0.0007749793887572033
Iteration: 34400, Loss: 0.000533231859096869
Iteration: 34500, Loss: 0.00027630928411229405
Iteration: 34600, Loss: 0.0005523899857891473
Iteration: 34700, Loss: 0.0007680635219792185
Iteration: 34800, Loss: 0.0005112358069213357
Iteration: 34900, Loss: 0.00026870888445917747
Iteration: 35000, Loss: 0.0005385954661024369
Iteration: 35100, Loss: 0.0007618989493353978
Iteration: 35200, Loss: 0.0004897641254240829
Iteration: 35300, Loss: 0.00026191088825786616
Iteration: 35400, Loss: 0.0005236218986629814
Iteration: 35500, Loss: 0.0007562543904372704
Iteration: 35600, Loss: 0.0004689675977285579
Iteration: 35700, Loss: 0.00025569372391400993
Iteration: 35800, Loss: 0.0005074815372832534
Iteration: 35900, Loss: 0.0007513304953776847
Iteration: 36000, Loss: 0.0004481705815735769
```

```
Iteration: 36100, Loss: 0.00025031181048284265
Iteration: 36200, Loss: 0.0004897815686273462
Iteration: 36300, Loss: 0.0007466637944802967
Iteration: 36400, Loss: 0.00042807794542735784
Iteration: 36500, Loss: 0.0002457188383664287
Iteration: 36600, Loss: 0.00046986524146127743
Iteration: 36700, Loss: 0.0007422889619116132
Iteration: 36800, Loss: 0.00040838856394468883
Iteration: 36900, Loss: 0.00024184124216760212
Iteration: 37000, Loss: 0.0004487657749022043
Iteration: 37100, Loss: 0.0007381707237813787
Iteration: 37200, Loss: 0.0003888425690221424
Iteration: 37300, Loss: 0.00023897857653442961
Iteration: 37400, Loss: 0.0004258836765893986
Iteration: 37500, Loss: 0.000733724457322704
Iteration: 37600, Loss: 0.0003695385450251366
Iteration: 37700, Loss: 0.00023713470946288285
Iteration: 37800, Loss: 0.0004022946114094985
Iteration: 37900, Loss: 0.0007287144768414986
Iteration: 38000, Loss: 0.0003503389235643406
Iteration: 38100, Loss: 0.00023630232989914685
Iteration: 38200, Loss: 0.0003784695540885154
Iteration: 38300, Loss: 0.0007232697570963819
Iteration: 38400, Loss: 0.0003312486708726404
Iteration: 38500, Loss: 0.00023656866427427848
Iteration: 38600, Loss: 0.00035559744170938617
Iteration: 38700, Loss: 0.0007172167830399467
Iteration: 38800, Loss: 0.0003126623146574749
Iteration: 38900, Loss: 0.00023765698317060056
Iteration: 39000, Loss: 0.00033426074234353594
Iteration: 39100, Loss: 0.0007109692606041567
Iteration: 39200, Loss: 0.00029516293106284437
Iteration: 39300, Loss: 0.00023694925691342511
Iteration: 39400, Loss: 0.000315074225992839
Iteration: 39500, Loss: 0.0007051207377137692
Iteration: 39600, Loss: 0.00027927054028072225
Iteration: 39700, Loss: 0.00024211986937989558
Iteration: 39800, Loss: 0.000298811667033089
Iteration: 39900, Loss: 0.00069944003695805
Iteration: 40000, Loss: 0.00026396016316650395
Iteration: 40100, Loss: 0.0002449800135056699
Iteration: 40200, Loss: 0.00028160716812985776
Iteration: 40300, Loss: 0.0006940005075007722
Iteration: 40400, Loss: 0.00025179430818218777
Iteration: 40500, Loss: 0.00024816468802568211
Iteration: 40600, Loss: 0.00026823405061919790
Iteration: 40700, Loss: 0.0006913649199239530
Iteration: 40800, Loss: 0.00024059840015265037
Iteration: 40900, Loss: 0.0002524533681629623
Iteration: 41000, Loss: 0.00025708276471889785
Iteration: 41100, Loss: 0.0006885336232269121
Iteration: 41200, Loss: 0.00023098950528827142
Iteration: 41300, Loss: 0.0002568821877602448
Iteration: 41400, Loss: 0.00024722252200061186
Iteration: 41500, Loss: 0.0006866619937511480
Iteration: 41600, Loss: 0.00022318834553854840
Iteration: 41700, Loss: 0.00026171850835903757
Iteration: 41800, Loss: 0.00023926916450916575
Iteration: 41900, Loss: 0.0006864487133496935
Iteration: 42000, Loss: 0.000216525722685846
```

```
Iteration: 42100, Loss: 0.0002671413860506873
Iteration: 42200, Loss: 0.0002332412605563887
Iteration: 42300, Loss: 0.0006868188402679365
Iteration: 42400, Loss: 0.00021094228822455036
Iteration: 42500, Loss: 0.000272959992459199
Iteration: 42600, Loss: 0.00022832662610300947
Iteration: 42700, Loss: 0.0006877418192791841
Iteration: 42800, Loss: 0.00020661731206331337
Iteration: 42900, Loss: 0.0002790267473849428
Iteration: 43000, Loss: 0.0002246330080108186
Iteration: 43100, Loss: 0.0006896605253367963
Iteration: 43200, Loss: 0.000203256606435396
Iteration: 43300, Loss: 0.000285453919954165
Iteration: 43400, Loss: 0.00022171644684345363
Iteration: 43500, Loss: 0.0006914825825043121
Iteration: 43600, Loss: 0.0002006233457049171
Iteration: 43700, Loss: 0.00029219591766142046
Iteration: 43800, Loss: 0.0002196647318723064
Iteration: 43900, Loss: 0.0006939679525849065
Iteration: 44000, Loss: 0.0001989353125428889
Iteration: 44100, Loss: 0.00029921141563148866
Iteration: 44200, Loss: 0.00021837817518830904
Iteration: 44300, Loss: 0.0006970818501600775
Iteration: 44400, Loss: 0.00019757173755264104
Iteration: 44500, Loss: 0.0003066926166890899
Iteration: 44600, Loss: 0.00021809879305153813
Iteration: 44700, Loss: 0.0006993861210711538
Iteration: 44800, Loss: 0.0001966304468994021
Iteration: 44900, Loss: 0.00031418209463737675
Iteration: 45000, Loss: 0.0002169463542498572
Iteration: 45100, Loss: 0.0007013339683034223
Iteration: 45200, Loss: 0.00019644934482782988
Iteration: 45300, Loss: 0.00032188874160119494
Iteration: 45400, Loss: 0.00021636508716385615
Iteration: 45500, Loss: 0.0007037438810756888
Iteration: 45600, Loss: 0.00019656676572945764
Iteration: 45700, Loss: 0.0003299856166323206
Iteration: 45800, Loss: 0.0002160564351353937
Iteration: 45900, Loss: 0.000705450812847808
Iteration: 46000, Loss: 0.00019694571926118104
Iteration: 46100, Loss: 0.0003382109952908327
Iteration: 46200, Loss: 0.0002151430829379894
Iteration: 46300, Loss: 0.0007062796987427244
Iteration: 46400, Loss: 0.00019781057073861629
Iteration: 46500, Loss: 0.00034670060902844174
Iteration: 46600, Loss: 0.00021392469637750023
Iteration: 46700, Loss: 0.0007066882229893961
Iteration: 46800, Loss: 0.00019925890342243165
Iteration: 46900, Loss: 0.0003553333453608401
Iteration: 47000, Loss: 0.00021364148985677196
Iteration: 47100, Loss: 0.0007064378792638267
Iteration: 47200, Loss: 0.00020088513053656424
Iteration: 47300, Loss: 0.0003641203050693425
Iteration: 47400, Loss: 0.00021275399374671778
Iteration: 47500, Loss: 0.0007054977472956944
Iteration: 47600, Loss: 0.00020318748825103259
Iteration: 47700, Loss: 0.0003733299397406269
Iteration: 47800, Loss: 0.00021193824465756 41
Iteration: 47900, Loss: 0.0007044486788123793
Iteration: 48000, Loss: 0.000205906104334088
```

```
Iteration: 48100, Loss: 0.0003826162789049738
Iteration: 48200, Loss: 0.00021099321292793026
Iteration: 48300, Loss: 0.0007025005127419301
Iteration: 48400, Loss: 0.00020897021141834542
Iteration: 48500, Loss: 0.00039211563664070197
Iteration: 48600, Loss: 0.00020900598471527778
Iteration: 48700, Loss: 0.0007000045587380548
Iteration: 48800, Loss: 0.0002123124765729688
Iteration: 48900, Loss: 0.0004016461992593285
Iteration: 49000, Loss: 0.0002043157338815484
Iteration: 49100, Loss: 0.0006943616583163213
Iteration: 49200, Loss: 0.00021666194008266389
Iteration: 49300, Loss: 0.00041559876965163587
Iteration: 49400, Loss: 0.00020645120419477144
Iteration: 49500, Loss: 0.0007010387854763729
Iteration: 49600, Loss: 0.00022188584652512022
Iteration: 49700, Loss: 0.00042536591336989873
Iteration: 49800, Loss: 0.00021355083310152016
Iteration: 49900, Loss: 0.0007030561351827647
Iteration: 50000, Loss: 0.00022453292826979068
Iteration: 50100, Loss: 0.0004326921694866742
Iteration: 50200, Loss: 0.00022491073676725093
Iteration: 50300, Loss: 0.0006938248776616874
Iteration: 50400, Loss: 0.00022143462365154909
Iteration: 50500, Loss: 0.00043223020163692246
Iteration: 50600, Loss: 0.0001891579315670358
Iteration: 50700, Loss: 0.0006789035661709332
Iteration: 50800, Loss: 0.00021324262293364349
Iteration: 50900, Loss: 0.0004121690140694835
Iteration: 51000, Loss: 8.615382523679631e-5
Iteration: 51100, Loss: 0.000891342142327344
Iteration: 51200, Loss: 0.0013872000387326034
Iteration: 51300, Loss: 0.0006804243854403441
Iteration: 51400, Loss: 0.00023842087942409367
Iteration: 51500, Loss: 0.0004512903999241778
Iteration: 51600, Loss: 0.000377987326081985
Iteration: 51700, Loss: 0.00040585375531596507
Iteration: 51800, Loss: 0.00024031944791995662
Iteration: 51900, Loss: 0.0003966834479932179
Iteration: 52000, Loss: 0.00022467848197148366
Iteration: 52100, Loss: 0.000377666808837964
Iteration: 52200, Loss: 0.00018292386779263916
Iteration: 52300, Loss: 0.0003675517097753831
Iteration: 52400, Loss: 0.00015251344413900417
Iteration: 52500, Loss: 0.0003634702565409911 6
Iteration: 52600, Loss: 0.00013311279720332032
Iteration: 52700, Loss: 0.0003609841205499009
Iteration: 52800, Loss: 0.00014980497173894717
Iteration: 52900, Loss: 0.00044729737260640123
Iteration: 53000, Loss: 0.00048678279745884446
Iteration: 53100, Loss: 0.0005878038030835086
Iteration: 53200, Loss: 0.0003675550353207692
Iteration: 53300, Loss: 0.00043817843749620924
Iteration: 53400, Loss: 0.00018316760406858818
Iteration: 53500, Loss: 0.0005120820726528639
Iteration: 53600, Loss: 0.00011341533339821043
Iteration: 53700, Loss: 0.0005452387431055237
Iteration: 53800, Loss: 0.00011647291963572825
Iteration: 53900, Loss: 0.0005220498874128643
Iteration: 54000, Loss: 0.00016279801418277 12
```

```
Iteration: 54100, Loss: 0.0005520194649434131
Iteration: 54200, Loss: 0.0001737813842337859 5
Iteration: 54300, Loss: 0.000589658733173136
Iteration: 54400, Loss: 0.0002914397998316
Iteration: 54500, Loss: 0.0006422009224072405
Iteration: 54600, Loss: 0.0001762822157339763 5
Iteration: 54700, Loss: 0.000510924428476596
Iteration: 54800, Loss: 0.0002063256317663551 7
Iteration: 54900, Loss: 0.000556363599566109
Iteration: 55000, Loss: 0.0003878762701451181 3
Iteration: 55100, Loss: 0.0006211031512088987
Iteration: 55200, Loss: 0.0005069114981602281
Iteration: 55300, Loss: 0.0006096875664098355
Iteration: 55400, Loss: 0.0007495674581899231
Iteration: 55500, Loss: 0.0005943916682813216
Iteration: 55600, Loss: 0.0005350522472272393
Iteration: 55700, Loss: 0.0004768554422323332 6
Iteration: 55800, Loss: 0.0004619728325854879
Iteration: 55900, Loss: 0.0004577343962357421
Iteration: 56000, Loss: 0.0005324211483390512
Iteration: 56100, Loss: 0.0004673599506597286
Iteration: 56200, Loss: 0.0003291334427293184
Iteration: 56300, Loss: 0.0004588359289885995 3
Iteration: 56400, Loss: 0.0001152191271355592 2
Iteration: 56500, Loss: 0.0008301462174242286
Iteration: 56600, Loss: 8.828535955128273e-5
Iteration: 56700, Loss: 0.0005291215075229432
Iteration: 56800, Loss: 0.0001463830382542196 8
Iteration: 56900, Loss: 0.0005261642606650426
Iteration: 57000, Loss: 0.0001172009267183793 8
Iteration: 57100, Loss: 0.0005345674665198539
Iteration: 57200, Loss: 0.0001153292498240902
Iteration: 57300, Loss: 0.0009208001305266211
Iteration: 57400, Loss: 0.0003533421798333619 4
Iteration: 57500, Loss: 0.0005186162294764738
Iteration: 57600, Loss: 9.907126290931417e-5
Iteration: 57700, Loss: 0.0006756052231935993
Iteration: 57800, Loss: 0.0001276953201575485 2
Iteration: 57900, Loss: 0.0004963976790099254
Iteration: 58000, Loss: 0.0001882134672555529
Iteration: 58100, Loss: 0.0008149268217087438
Iteration: 58200, Loss: 0.0007689991240525929
Iteration: 58300, Loss: 0.0006499399500217182
Iteration: 58400, Loss: 0.0003057432562123993
Iteration: 58500, Loss: 0.0005822490272570099
Iteration: 58600, Loss: 0.0001730127591667549 4
Iteration: 58700, Loss: 0.0004514274283868966
Iteration: 58800, Loss: 0.0001309052186170142 3
Iteration: 58900, Loss: 0.0005775311014312741
Iteration: 59000, Loss: 7.43611155508162e-5
Iteration: 59100, Loss: 0.0005434069925921196
Iteration: 59200, Loss: 0.0001312053022691735 2
Iteration: 59300, Loss: 0.0006489701109798334
Iteration: 59400, Loss: 0.0001427927257805708 3
Iteration: 59500, Loss: 0.0014590249639630031
Iteration: 59600, Loss: 0.000165234564232359
Iteration: 59700, Loss: 0.0006935244564277727
Iteration: 59800, Loss: 0.0007271768511446631
Iteration: 59900, Loss: 0.0005841573779081214
Iteration: 60000, Loss: 0.0002137889417297792
```

```
Iteration: 60100, Loss: 0.0004436843215718264
Iteration: 60200, Loss: 0.0001518379020716966 3
Iteration: 60300, Loss: 0.0004322674313788407 6
Iteration: 60400, Loss: 0.0001478392995484281 7
Iteration: 60500, Loss: 0.0004270559530945310 6
Iteration: 60600, Loss: 0.000187487651681645 3
Iteration: 60700, Loss: 0.0004417940797809142 6
Iteration: 60800, Loss: 0.000353766525873862 9
Iteration: 60900, Loss: 0.001051292594546614
Iteration: 61000, Loss: 0.0002469422116697968 7
Iteration: 61100, Loss: 0.000633088336686860 6
Iteration: 61200, Loss: 0.0001487899223687799 3
Iteration: 61300, Loss: 0.000496696092487762 7
Iteration: 61400, Loss: 0.000172146113655047 3
Iteration: 61500, Loss: 0.000489090228322475 8
Iteration: 61600, Loss: 0.0001352699464886497 5
Iteration: 61700, Loss: 0.000491862811470575 9
Iteration: 61800, Loss: 0.0001483708025247474 7
Iteration: 61900, Loss: 0.000502269722835509 5
Iteration: 62000, Loss: 0.0001579579105384870 2
Iteration: 62100, Loss: 0.000541585399673158 3
Iteration: 62200, Loss: 0.000912039974846191 6
Iteration: 62300, Loss: 0.000976664599077619 2
Iteration: 62400, Loss: 0.000391913823310935 3
Iteration: 62500, Loss: 0.000666755511391790 3
Iteration: 62600, Loss: 0.000560315686635769 1
Iteration: 62700, Loss: 0.000732962702982998 5
Iteration: 62800, Loss: 0.0002750426283000830 5
Iteration: 62900, Loss: 0.000506256262818020 2
Iteration: 63000, Loss: 0.000232934777151210 8
Iteration: 63100, Loss: 0.000528071202276214
Iteration: 63200, Loss: 0.0001864131347653357 4
Iteration: 63300, Loss: 0.000515527850161586 7
Iteration: 63400, Loss: 0.0002069615467900494 6
Iteration: 63500, Loss: 0.000506899408927394 9
Iteration: 63600, Loss: 0.0001310078149030421 1
Iteration: 63700, Loss: 0.000500870030728746 2
Iteration: 63800, Loss: 0.00015423013179345
Iteration: 63900, Loss: 0.000544409138110850 6
Iteration: 64000, Loss: 0.0002614435226195821 4
Iteration: 64100, Loss: 0.000580101810715856 9
Iteration: 64200, Loss: 0.0003033469494306731 5
Iteration: 64300, Loss: 0.000574275846396419 5
Iteration: 64400, Loss: 0.0002890376543013887 5
Iteration: 64500, Loss: 0.000571670942525641 3
Iteration: 64600, Loss: 0.0002684803568323549 3
Iteration: 64700, Loss: 0.000576286543180735 3
Iteration: 64800, Loss: 0.0003324626999363420 7
Iteration: 64900, Loss: 0.000578365809072266 2
Iteration: 65000, Loss: 0.000509970931585624 8
Iteration: 65100, Loss: 0.000630060456627183 8
Iteration: 65200, Loss: 0.000222620130507816 3
Iteration: 65300, Loss: 0.000966054231573800 8
Iteration: 65400, Loss: 0.0002645961428104119 6
Iteration: 65500, Loss: 0.000632875654794013 5
Iteration: 65600, Loss: 0.000679078511275400 7
Iteration: 65700, Loss: 0.000569273666730515 8
Iteration: 65800, Loss: 0.0001410148823161818 3
Iteration: 65900, Loss: 0.000518012209764264 9
Iteration: 66000, Loss: 0.000265857049415529
```

```
Iteration: 66100, Loss: 0.0005108212403252363
Iteration: 66200, Loss: 0.00011521824089036399
Iteration: 66300, Loss: 0.0004810387807565 1254
Iteration: 66400, Loss: 0.0001680220668781867
Iteration: 66500, Loss: 0.0005033306637810476
Iteration: 66600, Loss: 0.00026273560089222335
Iteration: 66700, Loss: 0.0005560015179334766
Iteration: 66800, Loss: 0.0003142555787436668
Iteration: 66900, Loss: 0.0005643979054273782
Iteration: 67000, Loss: 0.0002796952922405974
Iteration: 67100, Loss: 0.0005623268454359889
Iteration: 67200, Loss: 0.00013929545806692022
Iteration: 67300, Loss: 0.0006150517793457104
Iteration: 67400, Loss: 0.0004141436026207506
Iteration: 67500, Loss: 0.001195273698969408
Iteration: 67600, Loss: 0.0006011044743968777
Iteration: 67700, Loss: 0.0006209040123943912
Iteration: 67800, Loss: 0.0004915153379746558
Iteration: 67900, Loss: 0.0005915975925328969
Iteration: 68000, Loss: 0.00020203030958864158
Iteration: 68100, Loss: 0.0005387546944902693
Iteration: 68200, Loss: 0.0003638394898814175
Iteration: 68300, Loss: 0.0005670097624709589
Iteration: 68400, Loss: 9.222983828069395e-5
Iteration: 68500, Loss: 0.0005400159230609945
Iteration: 68600, Loss: 0.00031157423337800557
Iteration: 68700, Loss: 0.000567492746463301
Iteration: 68800, Loss: 8.035188270171538e-5
Iteration: 68900, Loss: 0.0005259402002539672
Iteration: 69000, Loss: 0.0001321132693882529
Iteration: 69100, Loss: 0.00048464302945274855
Iteration: 69200, Loss: 0.00015907146344068662
Iteration: 69300, Loss: 0.0004893508385759556
Iteration: 69400, Loss: 0.00033465646478703835
Iteration: 69500, Loss: 0.0005948897237151173
Iteration: 69600, Loss: 0.00046181629302354275
Iteration: 69700, Loss: 0.0005874294069402071
Iteration: 69800, Loss: 0.00040902882325557843
Iteration: 69900, Loss: 0.0005588940052774206
Iteration: 70000, Loss: 0.0002456212262860981
Iteration: 70100, Loss: 0.0009473634439775895
Iteration: 70200, Loss: 0.002975786475105521
Iteration: 70300, Loss: 0.0008014385276569744
Iteration: 70400, Loss: 0.0022895963739959146
Iteration: 70500, Loss: 0.0008044111317535732
Iteration: 70600, Loss: 0.00031570775231220475
Iteration: 70700, Loss: 0.0005896522891357235
Iteration: 70800, Loss: 0.0003016319750689177
Iteration: 70900, Loss: 0.000603342557337806
Iteration: 71000, Loss: 0.00022958789020591493
Iteration: 71100, Loss: 0.0005927416393380944
Iteration: 71200, Loss: 0.00022379563887525765
Iteration: 71300, Loss: 0.0005834409462591587
Iteration: 71400, Loss: 0.00018453844527136922
Iteration: 71500, Loss: 0.0005651603964886322
Iteration: 71600, Loss: 0.00017090647233256284
Iteration: 71700, Loss: 0.0005659066853810533
Iteration: 71800, Loss: 0.00015767551210001221
Iteration: 71900, Loss: 0.0005600394854358079
Iteration: 72000, Loss: 0.00020493277038315355
```

```
Iteration: 72100, Loss: 0.0005647038564631706
Iteration: 72200, Loss: 0.0002515037815156362
Iteration: 72300, Loss: 0.0005718269194097176
Iteration: 72400, Loss: 0.0002647402417272973
Iteration: 72500, Loss: 0.000571069080715578
Iteration: 72600, Loss: 0.0002526893374901995 3
Iteration: 72700, Loss: 0.0005851419025712501
Iteration: 72800, Loss: 0.0001998089446861013
Iteration: 72900, Loss: 0.000581365413151004
Iteration: 73000, Loss: 0.00017155457749836855
Iteration: 73100, Loss: 0.0006311168354262688
Iteration: 73200, Loss: 7.066640250361433e-5
Iteration: 73300, Loss: 0.001161487163095386
Iteration: 73400, Loss: 0.0008009342696009697
Iteration: 73500, Loss: 0.0006853105860736545
Iteration: 73600, Loss: 0.0009757016178192669
Iteration: 73700, Loss: 0.0005577989875959149
Iteration: 73800, Loss: 0.0015434540258953938
Iteration: 73900, Loss: 0.0006054184873968773
Iteration: 74000, Loss: 0.002067268504865594
Iteration: 74100, Loss: 0.0008896063070013455
Iteration: 74200, Loss: 0.002977510031922624
Iteration: 74300, Loss: 0.0009748615487162458
Iteration: 74400, Loss: 0.004727620052999008
Iteration: 74500, Loss: 0.001497298895642041
Iteration: 74600, Loss: 0.0004949423898279336
Iteration: 74700, Loss: 0.0010230868943751335
Iteration: 74800, Loss: 0.0007243116765621712
Iteration: 74900, Loss: 0.0009525441133403771
Iteration: 75000, Loss: 0.0009381743869196174
Iteration: 75100, Loss: 0.0008975767121769003
Iteration: 75200, Loss: 0.0010670897397823888
Iteration: 75300, Loss: 0.0008455193102738331
Iteration: 75400, Loss: 0.0010457834651169035
Iteration: 75500, Loss: 0.0008000895551139799
Iteration: 75600, Loss: 0.0011898784631472147
Iteration: 75700, Loss: 0.0008481748436756959
Iteration: 75800, Loss: 0.0006486497587835804
Iteration: 75900, Loss: 0.0007095456778652878
Iteration: 76000, Loss: 0.0002715370441615992
Iteration: 76100, Loss: 0.0006151961797236604
Iteration: 76200, Loss: 0.0001918632062369463
Iteration: 76300, Loss: 0.0006059129388613641
Iteration: 76400, Loss: 0.00019040263050061046
Iteration: 76500, Loss: 0.0006153676739209668
Iteration: 76600, Loss: 0.00021832212930121057
Iteration: 76700, Loss: 0.0006265271466430941
Iteration: 76800, Loss: 0.00026327380568677534
Iteration: 76900, Loss: 0.000634928334319178
Iteration: 77000, Loss: 0.00029964303304651804
Iteration: 77100, Loss: 0.0006447185044470675
Iteration: 77200, Loss: 0.00032378272667863523
Iteration: 77300, Loss: 0.0006479393686808882
Iteration: 77400, Loss: 0.0002611948554646421
Iteration: 77500, Loss: 0.0006571698755909039
Iteration: 77600, Loss: 0.00020598418019890174
Iteration: 77700, Loss: 0.0007965821426740571
Iteration: 77800, Loss: 0.0014951869836859844
Iteration: 77900, Loss: 0.0012131804085471003
Iteration: 78000, Loss: 0.00107177882217803
```

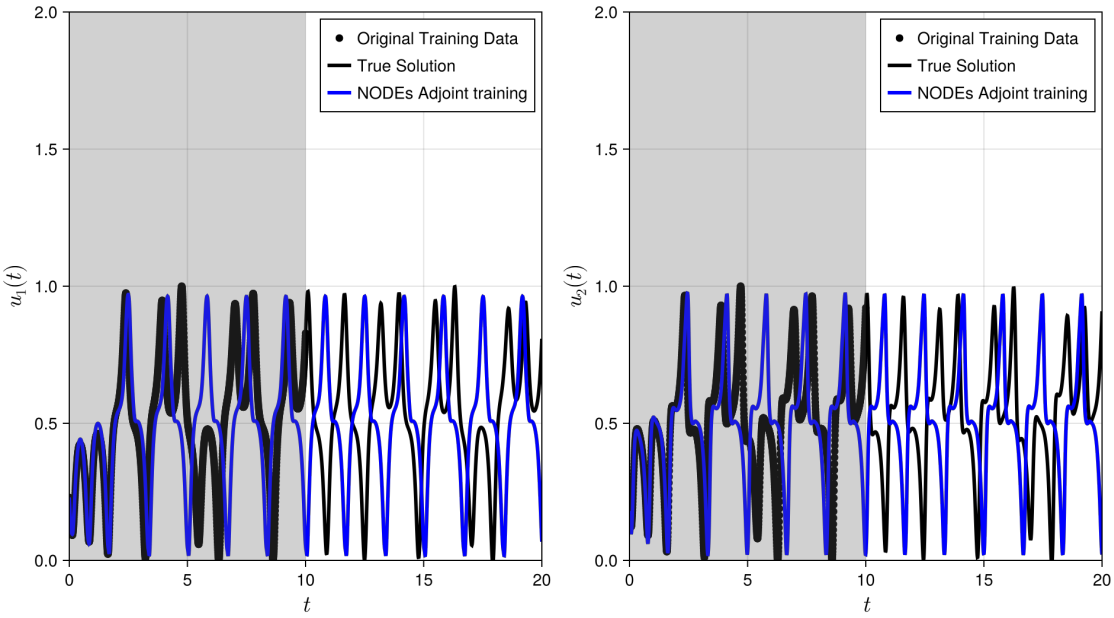
```
Iteration: 78100, Loss: 0.0006087315518428164
Iteration: 78200, Loss: 0.0001782311510906153 3
Iteration: 78300, Loss: 0.000525565740233918
Iteration: 78400, Loss: 0.0001402101031755571 7
Iteration: 78500, Loss: 0.0004791619672200735
Iteration: 78600, Loss: 0.0002124454335805921 3
Iteration: 78700, Loss: 0.0004868905237583571
Iteration: 78800, Loss: 0.0001801199933209484 6
Iteration: 78900, Loss: 0.0005384705128564662
Iteration: 79000, Loss: 0.0001280586485323305 8
Iteration: 79100, Loss: 0.000722470759388253
Iteration: 79200, Loss: 0.0008377874712382442
Iteration: 79300, Loss: 0.0007558050963572383
Iteration: 79400, Loss: 7.993531308594274e-5
Iteration: 79500, Loss: 0.0005229399769271085
Iteration: 79600, Loss: 0.0007797833200288743
Iteration: 79700, Loss: 0.0007392516190431165
Iteration: 79800, Loss: 0.0002420009500698151 7
Iteration: 79900, Loss: 0.0006096746091566466
Iteration: 80000, Loss: 0.0002286552025958588 4
Iteration: 80000, Loss: 5.70292372682085e-6
80001-element Vector{Float64}:
 0.0005582111988671 63
 0.0135865398379795 33
 0.0020680467565669 02
 0.0007722323799564 3182
 0.0074296482507452 01
 0.0027517896557092 11
 0.0018449252597159 73
 0.0007576937711369 859
 0.0029670031092201 587
 0.0018135121601798 41
 ⋮
 3.1440411495810697e-5
 5.887416166010757e-5
 0.0004753631522789 7125
 0.0001475403483786 8523
 0.0001044049114260 7579
 0.0004444843132921667
 5.82238050408169e-5
 0.0002286552025958588 4
 5.70292372682085e-6
```

Predict using NODEs solution and plot

In [21]:

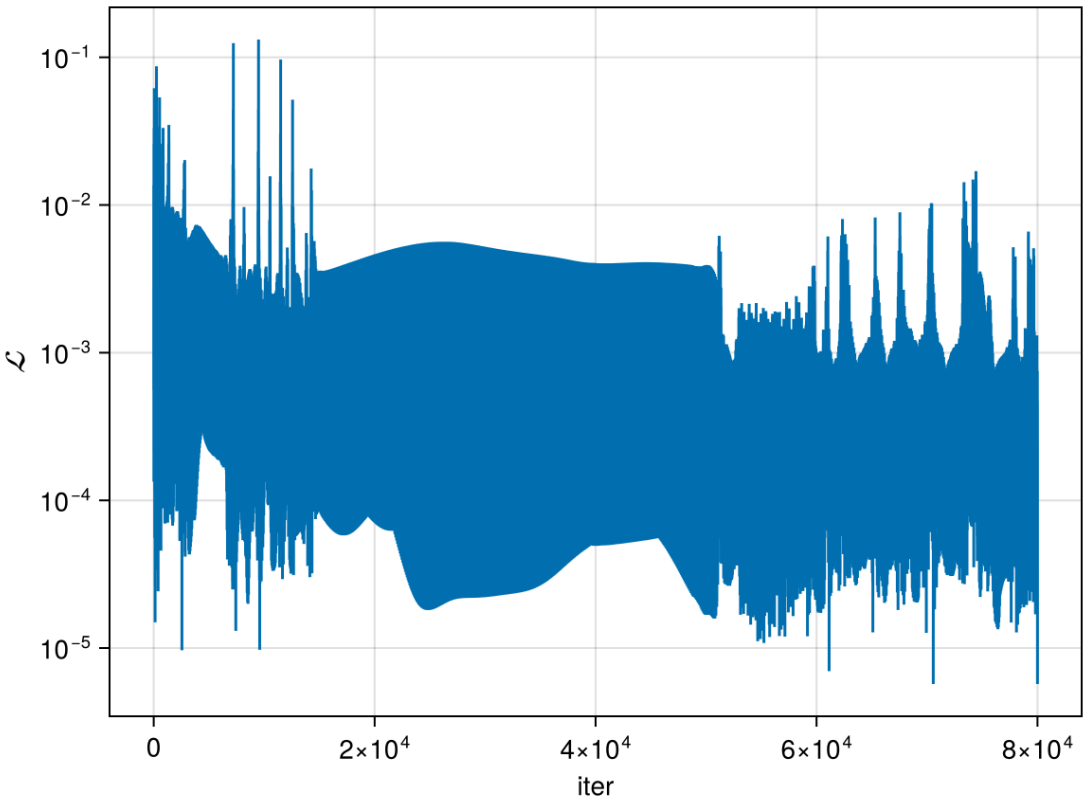
```
trained_model=StatefulLuxLayer(dudtNODEs, numerical_neuralode.u, smodel.s
Finaldudt(u, p, tplot) = trained_model(u, p)
prob = ODEProblem(Finaldudt, u0, (tspan2[1],tspan2[2]), numerical_neuralo
nn_sol = solve(prob, Tsit5(); saveat=tsteps2)


ComputedSolution_nodes=convert(AbstractArray,nn_sol)


fig = CairoMakie.Figure(size = (960, 540), backgroundcolor = :transparent
ax1 = CairoMakie.Axis(fig[1, 1],
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
    xlabel=L"t",ylabel=L"u_1(t)")
```

```
CairoMakie.scatter!(ax1,tsteps, data[1,:];color=:black,label="Original Tr
CairoMakie.lines!(ax1,tsteps2, data_true[1,:]; linewidth = 3,color=:black
CairoMakie.lines!(ax1,nn_sol_colloc.t,ComputedSolution_nodes[1,:]; linewi
CairoMakie.vspan!(ax1,tspan[1], tspan[2];color=(:grey, 0.2))
axislegend(ax1;position=:rt)
xlims!(ax1,tspan2[1],tspan2[2])
ylims!(ax1,0,2)

ax2 = CairoMakie.Axis(fig[1, 2],
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
    xlabel=L"t",ylabel=L"u_2(t)")
CairoMakie.scatter!(ax2,tsteps, data[2,:];color=:black,label="Original Tr
CairoMakie.lines!(ax2,tsteps2, data_true[2,:]; linewidth = 3,color=:black
CairoMakie.lines!(ax2,nn_sol_colloc.t,ComputedSolution_nodes[2,:]; linewi
CairoMakie.vspan!(ax2,tspan[1], tspan[2];color=(:grey, 0.2))
axislegend(ax2;position=:rt)
xlims!(ax2,tspan2[1],tspan2[2])
ylims!(ax2,0,2)
fig
```



Plot loss history

In [26]:
```
fig = CairoMakie.Figure()
ax = CairoMakie.Axis(fig[1, 1],yscale=log10,xlabel="iter",ylabel=L"\mathc
CairoMakie.lines!(ax,loss_history_NODEs)
fig
```



Unscale the data

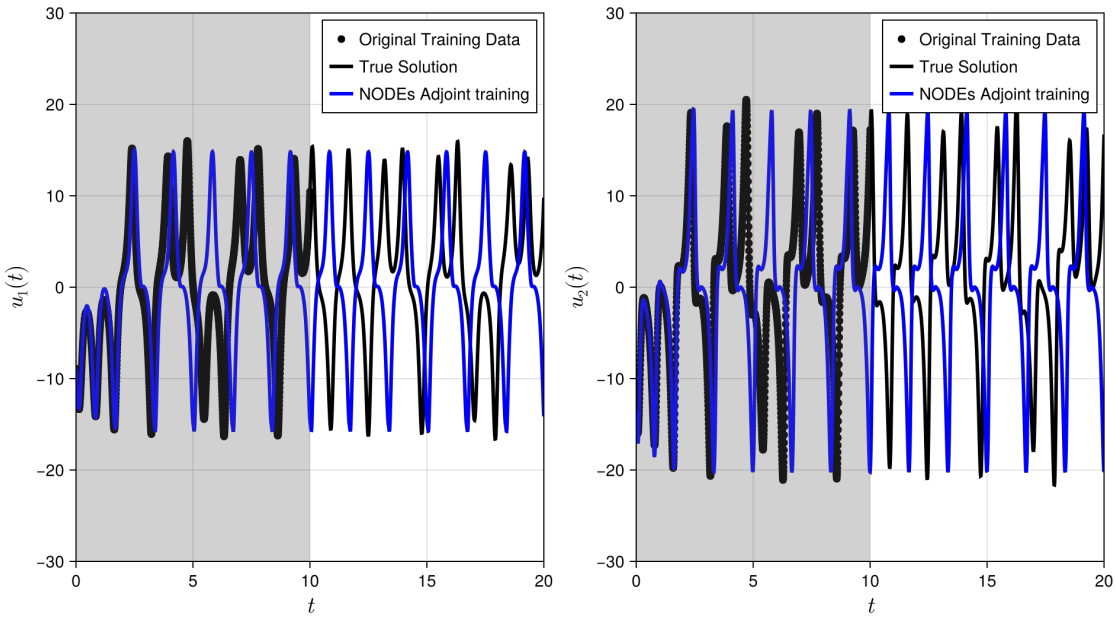
In [23]:
```
data=data.*(max_data .- min_data) .+ min_data
data_true=data_true.*(max_data .- min_data) .+ min_data
```

```
3×2000 Matrix{Float64}:
  -8.90545   -9.37906   -9.85893  -10.3395  …   8.47334   9.12364   9.8023
5
 -13.5921   -14.1525   -14.6693   -15.1259     14.8112   15.7711   16.7093
  19.9986    20.7246    21.5483    22.4675     14.8869   15.8234   16.9244
```

Unscale the predictions and plot with original data

In [ ]:
```
prob = ODEProblem(Finaldudt, u0, (tspan2[1],tspan2[2]), numerical_neuralo
#prob = ODEProblem(Finaldudt, u0, (tspan2[1],tspan2[2]), pinit)
nn_sol = solve(prob, Tsit5(); saveat=tsteps2)

ComputedSolution_nodes=convert(AbstractArray,nn_sol)

#unscale the computed solution
ComputedSolution_nodes=ComputedSolution_nodes.*(max_data .- min_data) .+

fig = CairoMakie.Figure(size = (960, 540), backgroundcolor = :transparent
ax1 = CairoMakie.Axis(fig[1, 1],
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
    xlabel=L"t",ylabel=L"u_1(t)")

CairoMakie.scatter!(ax1,tsteps, data[1,:];color=:black,label="Original Tr
CairoMakie.lines!(ax1,tsteps2, data_true[1,:]; linewidth = 3,color=:black
CairoMakie.lines!(ax1,nn_sol_colloc.t,ComputedSolution_nodes[1,:]; linewi
CairoMakie.vspan!(ax1,tspan[1], tspan[2];color=(:grey, 0.2))
```

```
axislegend(ax1;position=:rt)
xlims!(ax1,tspan2[1],tspan2[2])
ylims!(ax1,-30,30)

ax2 = CairoMakie.Axis(fig[1, 2],
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
    xlabel=L"t",ylabel=L"u_2(t)")
CairoMakie.scatter!(ax2,tsteps, data[2,:];color=:black,label="Original Tr
CairoMakie.lines!(ax2,tsteps2, data_true[2,:]; linewidth = 3,color=:black
CairoMakie.lines!(ax2,nn_sol_colloc.t,ComputedSolution_nodes[2,:]; linewi
CairoMakie.vspan!(ax2,tspan[1], tspan[2];color=(:grey, 0.2))
axislegend(ax2;position=:rt)
xlims!(ax2,tspan2[1],tspan2[2])
ylims!(ax2,-30,30)
save("figures/08LorenzResultNODEs.png", fig)
fig
```



In [25]:
```
using JLD2
ps_adjoint = numerical_neuralode.u
ps_colloc = result_neuralode.u

@save "trained_model.jld2" dudtNODEs  ps_adjoint ps_colloc  st  u0 tsteps
```